

Training a Playlist Curator Based on User Taste

UE24CS352A Machine Learning · Mini-Project · Problem No. 26 | Rekha Dhorigol (PES1UG24CS370), Sai Manogna B V (PES1UG24CS399)

1. Problem statement

Given a set of playlists and a new, unclassified song, place the song in the best-matching playlist – or suggest that a **new playlist** be created. We use the Stanford paper *Training a Playlist Curator Based on User Taste* (Awadelkarim & Coelho, 2018) as the baseline, re-implement its methods, and extend them with new features, more models, hyperparameter tuning, feature selection, error analysis and an interactive app.

2. Dataset

We use the Kaggle dataset *30000 Spotify Songs* (32,833 tracks, 23 columns, 449 playlists). We selected 13 distinct playlists (3,690 tracks), balanced them, removed duplicate `track_ids` and kept **52 songs per playlist** → **676 songs** (chance accuracy 7.7%). Features are the paper's 10 Spotify audio features (set A), plus 3 engineered ratios (set B: energy/danceability, acousticness/energy, tempo×energy), plus instrumentality, duration, track popularity and release year (set C). The paper's 2,095 one-hot genre tags are not available in this dataset, and `playlist_genre/subgenre` were deliberately *not* used because they derive from the label. We split 80/20 (stratified): 540 training and 136 test songs.

3. Approach

- Baseline (paper):** Perceptron, polynomial/sigmoid/RBF SVM and a 1-hidden-layer neural network, with and without feature scaling, 5-fold CV.
- Our additions:** feature sets A/B/C; Logistic Regression, KNN, Random Forest and HistGradientBoosting; a soft-voting ensemble; **hyperparameter tuning** with GridSearchCV (SVM) and RandomizedSearchCV (Random Forest, MLP, HistGB), where the scaler itself (Standard vs Quantile) is also tuned; a validation curve for the SVM penalty C; repeated stratified 5-fold CV (×3); an **ablation study** and a **segmentation study** (accuracy vs number of playlists); **feature selection** with mutual information (SelectKBest); and a tuned **abstain threshold** that implements the paper's "or suggest a new playlist" requirement.
- No leakage:** scaling and feature selection run inside sklearn Pipelines, all tuning uses only the training split, and the final model is chosen by cross-validation, not by the test set.

4. Implementation overview

The Colab notebook has 16 sections: data loading and EDA, cleaning and balancing, preprocessing and feature engineering, train/test split, baselines, tuning, repeated CV, feature selection, model comparison, error analysis (confusion matrix, PCA), permutation importance, abstain-threshold tuning, saving the final model (`joblib`) and a prediction demo. A Streamlit app (`app.py`) loads the saved model: the user sets a song's features and gets the top-3 playlists with probabilities, or a "create a new playlist" suggestion when confidence is below the threshold. Code: private GitHub repository (see README for setup).

5. Results

Model	Features	CV acc.	Test acc.	Macro-F1	Top-3
Baseline RBF SVM (paper)	A	0.294	0.301	0.286	0.632
Baseline NN, 1 hidden layer (paper)	A	0.310	0.301	0.282	0.574
SVM (tuned)	C	0.470	0.426	0.426	0.794
HistGradientBoosting (tuned)	C	0.523	0.515	0.508	0.801
Random Forest (tuned) – final	C	0.527	0.551	0.548	0.779

Table 1. CV = repeated 5-fold CV on the training split; test metrics on the held-out 136 songs.

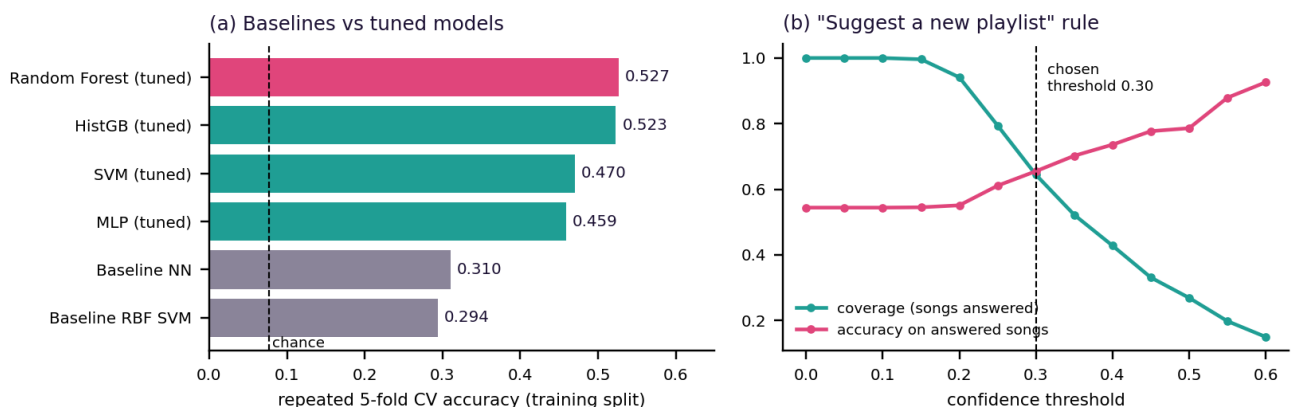


Figure 1. (a) Tuned models clearly beat the paper baselines. (b) Raising the confidence threshold trades coverage for accuracy.

- Replication:** we reproduce the paper's finding that feature scaling is essential (e.g. RBF SVM 0.274 scaled vs 0.104 unscaled). Our baseline (~0.30) is far below the paper's 0.77–0.82 because we have no genre tags and our playlists overlap more.
- Tuning helped:** SVM 0.409 → 0.469 CV (C=30, gamma=0.003, quantile scaling), HistGB 0.489 → 0.528, Random Forest 0.524 → 0.544. Mutual-information feature selection lifted the SVM to 0.494 CV (14 of 17 features).

- **Generalisation:** on 2,422 songs from the same playlists never seen in training the final model scores 0.460 accuracy, 0.430 macro-F1 and 0.741 top-3.
- **Abstain rule:** at threshold 0.30 the model answers 71% of test songs with 63.5% accuracy (55.1% when forced to answer all).
- **Drivers and errors:** release_year is by far the most important feature (permutation importance 0.27 vs 0.06 for the next). The ablation confirms it: without release year the tuned Random Forest drops from 0.527 to 0.434 CV, and audio features alone give 0.358. Most confusions are between near-duplicate playlists: Charts 2020 ↔ 2020 Hits and Neo-Soul ↔ Urban Contemporary.
- **Segmentation (idea from the paper):** accuracy rises as the number of playlists falls – Random Forest 0.54 with all 13 playlists, 0.73 with 5 and 0.86 with 2 – so a user with only a few playlists gets far better suggestions than the 13-class number suggests.

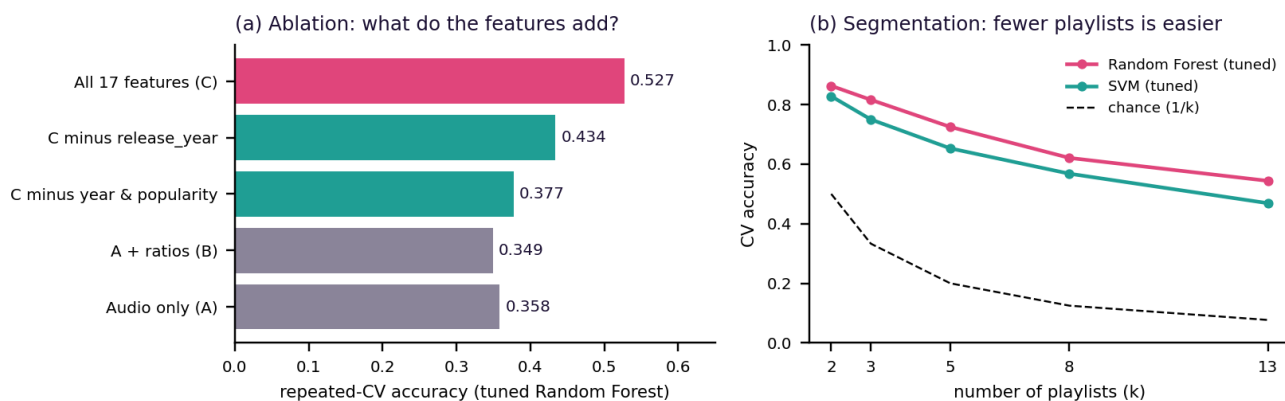


Figure 2. (a) Ablation: release year is the biggest single contributor, but metadata helps even without it. (b) Accuracy against the number of playlists (20 random subsets per k).

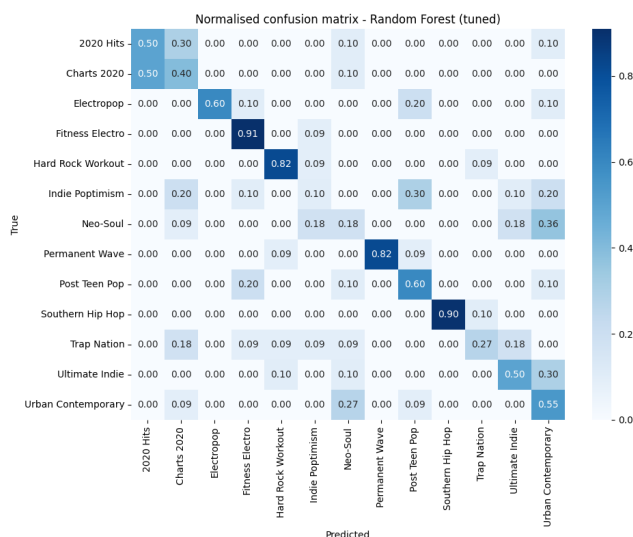


Figure 3. Normalised confusion matrix of the final model on the 136 test songs (about 10 per playlist, so single cells are noisy).

6. Conclusions

Starting from audio features alone, the paper's methods reach only ~30% accuracy on our 13 overlapping playlists. Adding metadata features, tuning hyperparameters and using tree ensembles raises this to **0.53 CV / 0.55 test accuracy** (0.78 top-3) – roughly seven times chance – and the abstain rule gives a practical way to say “this song needs a new playlist”. **Limitations:** small dataset, a small test split (± 4 points), overlapping playlists, and a model that leans heavily on release year (0.53 $\rightarrow$ 0.43 CV without it). **Future work:** genre/artist embeddings (node2vec, GloVe) as proposed by the paper, and larger per-user playlists.

7. Team contributions

Rekha Dhorigol: feature design and train/test split, hyperparameter tuning, model comparison and ablation, explainability, abstain-threshold tuning, final model, prediction demo, Streamlit app and README. **Sai Manogna B V:** data loading, EDA, cleaning and balancing, preprocessing and feature engineering, baseline replication, repeated cross-validation, feature selection, segmentation study and error analysis. Both members read the paper, tested the code and prepared the write-up and slides.

Reference

A. Awadelkarim and K. Coelho. *Training a Playlist Curator Based on User Taste*. Stanford University, 2018. Dataset: Kaggle, 30000 Spotify Songs (joebeachcapital).